

ENSF 480 - Lab 3

Source Code and Program Output

Sharar Masud, 30205753

Submission Date: 2026-09-28

Contents

Exercise A: 15 source files + program output

Exercise B: 3 source files + program output

Exercise C: 6 source files + program output

Exercise A

Source files

- shape.h
- shape.cpp
- point.h
- point.cpp
- rectangle.h
- rectangle.cpp
- square.h
- square.cpp
- circle.h
- circle.cpp
- curveCut.h
- curveCut.cpp
- graphicsWorld.h
- graphicsWorld.cpp
- main.cpp

Output: program_output.txt

Exercise A - File: shape.h

```
/*
 * File Name: shape.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#ifndef SHAPE_H
#define SHAPE_H

#include "point.h"

class Shape {
public:
    Shape(double x, double y, const char* name);
    Shape(const Shape& other);
    Shape& operator=(const Shape& other);
    virtual ~Shape();

    const Point& getOrigin() const;
    const char* getName() const;

    virtual double area() const;
    virtual double perimeter() const;
    virtual void display() const;

    double distance(const Shape& other) const;
    static double distance(const Shape& the_shape, const Shape& other);

    void move(double dx, double dy);

protected:
    Point origin;

private:
    char* shapeName;

    static char* duplicateName(const char* name);
};

#endif
```

Exercise A - File: shape.cpp

```
/*
 * File Name: shape.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#include "shape.h"

#include <cstring>
#include <iostream>

char* Shape::duplicateName(const char* name)
{
    const char* source = (name != nullptr) ? name : "";
    char* result = new char[std::strlen(source) + 1];
    std::strcpy(result, source);
    return result;
}

Shape::Shape(double x, double y, const char* name)
    : origin(x, y), shapeName(duplicateName(name))
{
}

Shape::Shape(const Shape& other)
    : origin(other.origin), shapeName(duplicateName(other.shapeName))
{
}

Shape& Shape::operator=(const Shape& other)
{
    if (this != &other) {
        char* newName = duplicateName(other.shapeName);
        origin = other.origin;
        delete[] shapeName;
        shapeName = newName;
    }
    return *this;
}

Shape::~Shape()
{
    delete[] shapeName;
}

const Point& Shape::getOrigin() const
{
    return origin;
}

const char* Shape::getName() const
{
    return shapeName;
}

double Shape::area() const
{
    return 0.0;
}

double Shape::perimeter() const
{
    return 0.0;
}

void Shape::display() const
{
    std::cout << "Shape Name: " << shapeName << '\n'
              << "X-coordinate: " << origin.getx() << '\n'
              << "Y-coordinate: " << origin.gety() << '\n';
}
```

Exercise A - File: shape.cpp (continued)

```
double Shape::distance(const Shape& other) const
{
    return origin.distance(other.origin);
}

double Shape::distance(const Shape& the_shape, const Shape& other)
{
    return Point::distance(the_shape.origin, other.origin);
}

void Shape::move(double dx, double dy)
{
    origin.setx(origin.getx() + dx);
    origin.sety(origin.gety() + dy);
}
```

Exercise A - File: point.h

```
/*
 * File Name: point.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#ifndef POINT_H
#define POINT_H

class Point {
public:
    Point(double x, double y);
    Point(const Point& other);
    Point& operator=(const Point& other);
    ~Point();

    double getx() const;
    double gety() const;
    int getid() const;

    void setx(double x);
    void sety(double y);

    void display() const;

    static int counter();

    double distance(const Point& other) const;
    static double distance(const Point& first, const Point& second);

private:
    double x;
    double y;
    int id;

    static int objectCount;
    static int nextId;
};

#endif
```

Exercise A - File: point.cpp

```
/*
 * File Name: point.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#include "point.h"

#include <cmath>
#include <iomanip>
#include <iostream>

int Point::objectCount = 0;
int Point::nextId = 1001;

Point::Point(double xValue, double yValue)
    : x(xValue), y(yValue), id(nextId++)
{
    ++objectCount;
}

Point::Point(const Point& other)
    : x(other.x), y(other.y), id(nextId++)
{
    ++objectCount;
}

Point& Point::operator=(const Point& other)
{
    if (this != &other) {
        x = other.x;
        y = other.y;
        // id deliberately remains unchanged: it identifies this Point object.
    }
    return *this;
}

Point::~Point()
{
    --objectCount;
}

double Point::getx() const
{
    return x;
}

double Point::gety() const
{
    return y;
}

int Point::getid() const
{
    return id;
}

void Point::setx(double xValue)
{
    x = xValue;
}

void Point::sety(double yValue)
{
    y = yValue;
}

void Point::display() const
{
    const std::ios::fmtflags oldFlags = std::cout.flags();
    const std::streamsize oldPrecision = std::cout.precision();
```

Exercise A - File: point.cpp (continued)

```
    std::cout << std::fixed << std::setprecision(2)
               << "X-coordinate: " << x << '\n'
               << "Y-coordinate: " << y << '\n';

    std::cout.flags(oldFlags);
    std::cout.precision(oldPrecision);
}

int Point::counter()
{
    return objectCount;
}

double Point::distance(const Point& other) const
{
    return Point::distance(*this, other);
}

double Point::distance(const Point& first, const Point& second)
{
    const double dx = first.x - second.x;
    const double dy = first.y - second.y;
    return std::sqrt(dx * dx + dy * dy);
}
```

Exercise A - File: rectangle.h

```
/*
 * File Name: rectangle.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#ifndef RECTANGLE_H
#define RECTANGLE_H

#include "square.h"

class Rectangle : public Square {
public:
    Rectangle(double x, double y, double sideA, double sideB, const char* name);
    Rectangle(const Rectangle& other);
    Rectangle& operator=(const Rectangle& other);

    double get_side_b() const;
    virtual void set_side_b(double sideB);

    double area() const override;
    double perimeter() const override;
    void display() const override;

private:
    double side_b;
};

#endif
```

Exercise A - File: rectangle.cpp

```
/*
 * File Name: rectangle.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#include "rectangle.h"

#include <iostream>

Rectangle::Rectangle(double x, double y, double sideA, double sideB, const char* name)
    : Shape(x, y, name), Square(x, y, sideA, name), side_b(sideB)
{
}

Rectangle::Rectangle(const Rectangle& other)
    : Shape(other), Square(other), side_b(other.side_b)
{
}

Rectangle& Rectangle::operator=(const Rectangle& other)
{
    if (this != &other) {
        Square::operator=(other);
        set_side_b(other.side_b);
    }
    return *this;
}

double Rectangle::get_side_b() const
{
    return side_b;
}

void Rectangle::set_side_b(double sideB)
{
    side_b = sideB;
}

double Rectangle::area() const
{
    return get_side_a() * side_b;
}

double Rectangle::perimeter() const
{
    return 2.0 * (get_side_a() + side_b);
}

void Rectangle::display() const
{
    std::cout << "Rectangle Name: " << getName() << '\n'
        << "X-coordinate: " << getOrigin().getX() << '\n'
        << "Y-coordinate: " << getOrigin().getY() << '\n'
        << "Side a: " << get_side_a() << '\n'
        << "Side b: " << side_b << '\n'
        << "Area: " << area() << '\n'
        << "Perimeter: " << perimeter() << '\n';
}
```

Exercise A - File: square.h

```
/*
 * File Name: square.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#ifndef SQUARE_H
#define SQUARE_H

#include "shape.h"

class Square : virtual public Shape {
public:
    Square(double x, double y, double sideA, const char* name);
    Square(const Square& other);
    Square& operator=(const Square& other);

    double get_side_a() const;
    virtual void set_side_a(double sideA);

    virtual double area() const;
    virtual double perimeter() const;
    void display() const override;

private:
    double side_a;
};

#endif
```

Exercise A - File: square.cpp

```
/*
 * File Name: square.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#include "square.h"

#include <iostream>

Square::Square(double x, double y, double sideA, const char* name)
    : Shape(x, y, name), side_a(sideA)
{
}

Square::Square(const Square& other)
    : Shape(other), side_a(other.side_a)
{
}

Square& Square::operator=(const Square& other)
{
    if (this != &other) {
        Shape::operator=(other);
        set_side_a(other.side_a);
    }
    return *this;
}

double Square::get_side_a() const
{
    return side_a;
}

void Square::set_side_a(double sideA)
{
    side_a = sideA;
}

double Square::area() const
{
    return side_a * side_a;
}

double Square::perimeter() const
{
    return 4.0 * side_a;
}

void Square::display() const
{
    std::cout << "Square Name: " << getName() << '\n'
        << "X-coordinate: " << getOrigin().getx() << '\n'
        << "Y-coordinate: " << getOrigin().gety() << '\n'
        << "Side a: " << side_a << '\n'
        << "Area: " << area() << '\n'
        << "Perimeter: " << perimeter() << '\n';
}
```

Exercise A - File: circle.h

```
/*
 * File Name: circle.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#ifndef CIRCLE_H
#define CIRCLE_H

#include "shape.h"

class Circle : virtual public Shape {
public:
    Circle(double x, double y, double radius, const char* name);
    Circle(const Circle& other);
    Circle& operator=(const Circle& other);

    double getRadius() const;
    virtual void setRadius(double radius);

    double area() const override;
    double perimeter() const override;
    void display() const override;

private:
    double radius;
};

#endif
```

Exercise A - File: circle.cpp

```
/*
 * File Name: circle.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#include "circle.h"

#include <cstdlib>
#include <iostream>

namespace {
const double PI = 3.14159265358979323846;

void validateRadius(double radius)
{
    if (radius < 0.0) {
        std::cerr << "Error: radius cannot be negative.\n";
        std::exit(EXIT_FAILURE);
    }
}

Circle::Circle(double x, double y, double radiusValue, const char* name)
    : Shape(x, y, name), radius(radiusValue)
{
    validateRadius(radius);
}

Circle::Circle(const Circle& other)
    : Shape(other), radius(other.radius)
{
}

Circle& Circle::operator=(const Circle& other)
{
    if (this != &other) {
        Shape::operator=(other);
        setRadius(other.radius);
    }
    return *this;
}

double Circle::getRadius() const
{
    return radius;
}

void Circle::setRadius(double radiusValue)
{
    validateRadius(radiusValue);
    radius = radiusValue;
}

double Circle::area() const
{
    return PI * radius * radius;
}

double Circle::perimeter() const
{
    return 2.0 * PI * radius;
}

void Circle::display() const
{
    std::cout << "Circle Name: " << getName() << '\n'
              << "X-coordinate: " << getOrigin().getx() << '\n'
              << "Y-coordinate: " << getOrigin().gety() << '\n'
              << "Radius: " << radius << '\n'
              << "Area: " << area() << '\n'
              << "Perimeter: " << perimeter() << '\n';
}
```

Exercise A - File: circle.cpp (continued)

```
}
```

Exercise A - File: curveCut.h

```
/*
 * File Name: curveCut.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#ifndef CURVECUT_H
#define CURVECUT_H

#include "circle.h"
#include "rectangle.h"

class CurveCut : public Rectangle, public Circle {
public:
    CurveCut(double x, double y, double width, double length,
             double radius, const char* name);
    CurveCut(const CurveCut& other);
    CurveCut& operator=(const CurveCut& other);

    void set_side_a(double width) override;
    void set_side_b(double length) override;
    void setRadius(double radius) override;

    double area() const override;
    double perimeter() const override;
    void display() const override;

private:
    void validateDimensions() const;
};

#endif
```

Exercise A - File: curveCut.cpp

```
/*
 * File Name: curveCut.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#include "curveCut.h"

#include <cstdlib>
#include <iostream>

namespace {
void terminateForInvalidDimensions()
{
    std::cerr << "Error: the radius of a CurveCut cannot exceed its width or length.\n";
    std::exit(EXIT_FAILURE);
}
}

CurveCut::CurveCut(double x, double y, double width, double length,
                  double radius, const char* name)
    : Shape(x, y, name),
      Rectangle(x, y, width, length, name),
      Circle(x, y, radius, name)
{
    validateDimensions();
}

CurveCut::CurveCut(const CurveCut& other)
    : Shape(other), Rectangle(other), Circle(other)
{
}

CurveCut& CurveCut::operator=(const CurveCut& other)
{
    if (this != &other) {
        Shape::operator=(other);
        Square::set_side_a(other.get_side_a());
        Rectangle::set_side_b(other.get_side_b());
        Circle::setRadius(other.getRadius());
    }
    return *this;
}

void CurveCut::set_side_a(double width)
{
    if (getRadius() > width) {
        terminateForInvalidDimensions();
    }
    Square::set_side_a(width);
}

void CurveCut::set_side_b(double length)
{
    if (getRadius() > length) {
        terminateForInvalidDimensions();
    }
    Rectangle::set_side_b(length);
}

void CurveCut::setRadius(double radius)
{
    if (radius > get_side_a() || radius > get_side_b()) {
        terminateForInvalidDimensions();
    }
    Circle::setRadius(radius);
}

double CurveCut::area() const
{
    return Rectangle::area() - Circle::area() / 4.0;
}
```

Exercise A - File: curveCut.cpp (continued)

```
double CurveCut::perimeter() const
{
    return Rectangle::perimeter() - 2.0 * getRadius()
        + Circle::perimeter() / 4.0;
}

void CurveCut::display() const
{
    std::cout << "CurveCut Name: " << getName() << '\n'
        << "X-coordinate: " << getOrigin().getX() << '\n'
        << "Y-coordinate: " << getOrigin().getY() << '\n'
        << "Width: " << get_side_a() << '\n'
        << "Length: " << get_side_b() << '\n'
        << "Radius of the cut: " << getRadius() << '\n';
}

void CurveCut::validateDimensions() const
{
    if (getRadius() > get_side_a() || getRadius() > get_side_b()) {
        terminateForInvalidDimensions();
    }
}
```

Exercise A - File: graphicsWorld.h

```
/*
 * File Name: graphicsWorld.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#ifndef GRAPHICSWORLD_H
#define GRAPHICSWORLD_H

class GraphicsWorld {
public:
    void run();
};

#endif
```

Exercise A - File: graphicsWorld.cpp

```
/*
 * File Name: graphicsWorld.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   Sharar Masud, 30205753
 * Submission Date: Sept. 21, 2026
 */

#include "graphicsWorld.h"

#include "circle.h"
#include "curveCut.h"
#include "point.h"
#include "rectangle.h"
#include "shape.h"
#include "square.h"

#include <iostream>

using namespace std;

void GraphicsWorld::run()
{
    cout << "Lab 3 - GraphicsWorld\nAuthor: Sharar\n";

    #if 1 // Point
        cout << "\nTesting Functions in class Point:" << endl;
        Point m(6, 8);
        Point n(6, 8);
        n.setX(9);
        cout << "Expected to display the distance between m and n is: 3";
        cout << "\nThe distance between m and n is: " << m.distance(n);
        cout << "\nExpected second version of the distance function also print: 3";
        cout << "\nThe distance between m and n is again: "
            << Point::distance(m, n) << endl;
    #endif

    #if 1 // Square
        cout << "\nTesting Functions in class Square:" << endl;
        Square s(5, 7, 12, "SQUARE - S");
        s.display();
    #endif

    #if 1 // Rectangle
        cout << "\nTesting Functions in class Rectangle:" << endl;
        Rectangle a(5, 7, 12, 15, "RECTANGLE A");
        a.display();

        Rectangle b(16, 7, 8, 9, "RECTANGLE B");
        b.display();

        double d = a.distance(b);
        cout << "\nDistance between rectangle a and b is: " << d << endl;

        Rectangle rec1 = a;
        rec1.display();

        cout << "\nTesting assignment operator in class Rectangle:" << endl;
        Rectangle rec2(3, 4, 11, 7, "RECTANGLE rec2");
        rec2.display();
        rec2 = a;
        a.set_side_b(200);
        a.set_side_a(100);

        cout << "\nExpected to display the following values for object rec2: " << endl;
        cout << "Rectangle Name: RECTANGLE A\n"
            << "X-coordinate: 5\n"
            << "Y-coordinate: 7\n"
            << "Side a: 12\n"
            << "Side b: 15\n"
            << "Area: 180\n"
            << "Perimeter: 54\n";
        cout << "\nIf it doesn't there is a problem with your assignment operator.\n" << endl;
        rec2.display();
    #endif
}
```

Exercise A - File: graphicsWorld.cpp (continued)

```
cout << "\nTesting copy constructor in class Rectangle:" << endl;
Rectangle rec3(a);
rec3.display();
a.set_side_b(300);
a.set_side_a(400);

cout << "\nExpected to display the following values for object rec3: " << endl;
cout << "Rectangle Name: RECTANGLE A\n"
    << "X-coordinate: 5\n"
    << "Y-coordinate: 7\n"
    << "Side a: 100\n"
    << "Side b: 200\n"
    << "Area: 20000\n"
    << "Perimeter: 600\n";
cout << "\nIf it doesn't there is a problem with your copy constructor.\n" << endl;
rec3.display();
#endif

#if 1 // Circle and CurveCut
cout << "\nTesting Functions in class Circle:" << endl;
Circle c(3, 5, 9, "CIRCLE C");
c.display();
cout << "The area of " << c.getName() << " is: " << c.area() << endl;
cout << "The perimeter of " << c.getName() << " is: " << c.perimeter() << endl;
cout << "The distance between rectangle a and circle c is: "
    << a.distance(c) << endl;

cout << "\nTesting Functions in class CurveCut:" << endl;
CurveCut rc(6, 5, 10, 12, 9, "CurveCut rc");
rc.display();
cout << "The area of " << rc.getName() << " is: " << rc.area() << endl;
cout << "The perimeter of " << rc.getName() << " is: " << rc.perimeter() << endl;
cout << "The distance between rc and c is: " << rc.distance(c) << endl;
#endif

#if 1 // Array of pointers and polymorphism
cout << "\nTesting array of Shape pointers and polymorphism:" << endl;
Shape* sh[4] = {&s, &a, &c, &rc};

for (Shape* shape : sh) {
    shape->display();
    cout << "The area of " << shape->getName() << " is: "
        << shape->area() << endl;
    cout << "The perimeter of " << shape->getName() << " is: "
        << shape->perimeter() << endl;
}
#endif

#if 1 // CurveCut copy constructor and assignment operator
cout << "\nTesting copy constructor in class CurveCut:" << endl;
CurveCut cc = rc;
cc.display();

cout << "\nTesting assignment operator in class CurveCut:" << endl;
CurveCut cc2(2, 5, 100, 12, 9, "CurveCut cc2");
cc2.display();
cc2 = cc;
cc2.display();
#endif
}
```

Exercise A - File: main.cpp

```
/*  
 * File Name: main.cpp  
 * Assignment: Lab 3 Exercise A  
 * Completed By:  
 *   Sharar Masud, 30205753  
 * Submission Date: Sept. 21, 2026  
 */
```

```
#include "graphicsWorld.h"
```

```
int main()  
{  
    GraphicsWorld world;  
    world.run();  
    return 0;  
}
```

Exercise A - Program Output (program_output.txt)

Lab 3 - GraphicsWorld
Author: Sharar

Testing Functions in class Point:
Expected to display the distance between m and n is: 3
The distance between m and n is: 3
Expected second version of the distance function also print: 3
The distance between m and n is again: 3

Testing Functions in class Square:
Square Name: SQUARE - S
X-coordinate: 5
Y-coordinate: 7
Side a: 12
Area: 144
Perimeter: 48

Testing Functions in class Rectangle:
Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 12
Side b: 15
Area: 180
Perimeter: 54
Rectangle Name: RECTANGLE B
X-coordinate: 16
Y-coordinate: 7
Side a: 8
Side b: 9
Area: 72
Perimeter: 34

Distance between rectangle a and b is: 11
Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 12
Side b: 15
Area: 180
Perimeter: 54

Testing assignment operator in class Rectangle:
Rectangle Name: RECTANGLE rec2
X-coordinate: 3
Y-coordinate: 4
Side a: 11
Side b: 7
Area: 77
Perimeter: 36

Expected to display the following values for object rec2:
Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 12
Side b: 15
Area: 180
Perimeter: 54

If it doesn't there is a problem with your assignment operator.

Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 12
Side b: 15
Area: 180
Perimeter: 54

Testing copy constructor in class Rectangle:
Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7

Exercise A - Program Output (program_output.txt) (continued)

Side a: 100
Side b: 200
Area: 20000
Perimeter: 600

Expected to display the following values for object rec3:

Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 100
Side b: 200
Area: 20000
Perimeter: 600

If it doesn't there is a problem with your copy constructor.

Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 100
Side b: 200
Area: 20000
Perimeter: 600

Testing Functions in class Circle:
Circle Name: CIRCLE C
X-coordinate: 3
Y-coordinate: 5
Radius: 9
Area: 254.469
Perimeter: 56.5487
The area of CIRCLE C is: 254.469
The perimeter of CIRCLE C is: 56.5487
The distance between rectangle a and circle c is: 2.82843

Testing Functions in class CurveCut:
CurveCut Name: CurveCut rc
X-coordinate: 6
Y-coordinate: 5
Width: 10
Length: 12
Radius of the cut: 9
The area of CurveCut rc is: 56.3827
The perimeter of CurveCut rc is: 40.1372
The distance between rc and c is: 3

Testing array of Shape pointers and polymorphism:

Square Name: SQUARE - S
X-coordinate: 5
Y-coordinate: 7
Side a: 12
Area: 144
Perimeter: 48
The area of SQUARE - S is: 144
The perimeter of SQUARE - S is: 48
Rectangle Name: RECTANGLE A
X-coordinate: 5
Y-coordinate: 7
Side a: 400
Side b: 300
Area: 120000
Perimeter: 1400
The area of RECTANGLE A is: 120000
The perimeter of RECTANGLE A is: 1400
Circle Name: CIRCLE C
X-coordinate: 3
Y-coordinate: 5
Radius: 9
Area: 254.469
Perimeter: 56.5487
The area of CIRCLE C is: 254.469
The perimeter of CIRCLE C is: 56.5487
CurveCut Name: CurveCut rc
X-coordinate: 6

Exercise A - Program Output (program_output.txt) (continued)

Y-coordinate: 5
Width: 10
Length: 12
Radius of the cut: 9
The area of CurveCut rc is: 56.3827
The perimeter of CurveCut rc is: 40.1372

Testing copy constructor in class CurveCut:
CurveCut Name: CurveCut rc
X-coordinate: 6
Y-coordinate: 5
Width: 10
Length: 12
Radius of the cut: 9

Testing assignment operator in class CurveCut:
CurveCut Name: CurveCut cc2
X-coordinate: 2
Y-coordinate: 5
Width: 100
Length: 12
Radius of the cut: 9
CurveCut Name: CurveCut rc
X-coordinate: 6
Y-coordinate: 5
Width: 10
Length: 12
Radius of the cut: 9

Exercise B

Source files

mystring2.h
mystring2.cpp
iterator.cpp

Output: output.txt

Exercise B - File: mystring2.h

```
//File: mystring2.h
// ENSF 480 - Lab 3

#ifndef MYSTRING_H
#define MYSTRING_H

#include <iostream>

class Mystring {
    friend std::ostream& operator <<(std::ostream& os, const Mystring& s);

public:
    Mystring();
    // PROMISES: Empty string object is created.

    Mystring(int n);
    // PROMISES: Creates an empty string with a total capacity of n.
    //           In other words, dynamically allocates n elements for
    //           charsM, sets lengthM to zero, and fills the first
    //           element of charsM with '\0'.

    Mystring(const char *s);
    // REQUIRES: s points to first char of a built-in string.
    // PROMISES: Mystring object is created by copying chars from s.

    ~Mystring();

    Mystring(const Mystring& source);

    Mystring& operator =(const Mystring& rhs);
    // REQUIRES: rhs is reference to a Mystring as a source.
    // PROMISES: Makes this object a copy of rhs.

    bool operator >(const Mystring& rhs) const;
    // PROMISES: Returns true if this string comes after rhs
    //           lexicographically.

    int length() const;
    // PROMISES: Return value is number of chars in charsM.

    char get_char(int pos) const;
    // REQUIRES: pos >= 0 && pos < length()
    // PROMISES: Return value is char at position pos.

    const char * c_str() const;
    // PROMISES: Return value points to first char in built-in string
    //           containing the chars of the string object.

    void set_char(int pos, char c);
    // REQUIRES: pos >= 0 && pos < length(), c != '\0'
    // PROMISES: Character at position pos is set equal to c.

    Mystring& append(const Mystring& other);
    // PROMISES: Concatenates other to the end of this string.

    void set_str(char* s);
    // REQUIRES: s is a valid C++ string of characters (a built-in string).
    // PROMISES: Copies s into charsM.

private:
    int lengthM;
    char* charsM;
    void memory_check(char* s);
};

#endif
```

Exercise B - File: mystring2.cpp

```
// mystring2.cpp
// ENSF 480 - Lab 3
// M. Moussavi

#include "mystring2.h"
#include <string.h>
#include <iostream>

using namespace std;

Mystring::Mystring()
{
    charsM = new char[1];
    charsM[0] = '\0';
    lengthM = 0;
}

Mystring::Mystring(const char *s)
    : lengthM((int)strlen(s))
{
    charsM = new char[lengthM + 1];
    strcpy(charsM, s);
}

Mystring::Mystring(int n)
    : lengthM(0), charsM(new char[n])
{
    charsM[0] = '\0';
}

Mystring::Mystring(const Mystring& source)
    : lengthM(source.lengthM), charsM(new char[source.lengthM + 1])
{
    strcpy(charsM, source.charsM);
}

Mystring::~Mystring()
{
    delete [] charsM;
}

int Mystring::length() const
{
    return lengthM;
}

char Mystring::get_char(int pos) const
{
    if(pos < 0 || pos >= length()) {
        cerr << "\nERROR: get_char: the position is out of boundary."
    }

    return charsM[pos];
}

const char * Mystring::c_str() const
{
    return charsM;
}

void Mystring::set_char(int pos, char c)
{
    if(pos < 0 || pos >= length()) {
        cerr << "\nset_char: the position is out of boundary."
        << " Nothing was changed.";
        return;
    }

    if(c == '\0') {
        cerr << "\nset_char: char c is empty."
        << " Nothing was changed.";
        return;
    }
}
```

Exercise B - File: mystring2.cpp (continued)

```
    charsM[pos] = c;
}

Mystring& Mystring::operator =(const Mystring& S)
{
    if(this == &S)
        return *this;

    delete [] charsM;
    lengthM = (int)strlen(S.charsM);
    charsM = new char[lengthM + 1];
    strcpy(charsM, S.charsM);
    return *this;
}

bool Mystring::operator >(const Mystring& rhs) const
{
    return strcmp(charsM, rhs.charsM) > 0;
}

Mystring& Mystring::append(const Mystring& other)
{
    char *tmp = new char[lengthM + other.lengthM + 1];
    strcpy(tmp, charsM);
    strcat(tmp, other.charsM);

    lengthM += other.lengthM;
    delete [] charsM;
    charsM = tmp;

    return *this;
}

void Mystring::set_str(char* s)
{
    delete [] charsM;
    lengthM = (int)strlen(s);
    charsM = new char[lengthM + 1];
    strcpy(charsM, s);
}

std::ostream& operator <<(std::ostream& os, const Mystring& s)
{
    return os << s.charsM;
}
```

Exercise B - File: iterator.cpp

```
// iterator.cpp
// ENSF 480 - Fall 2022 - Lab 3, Ex B

#include <iostream>
#include <cassert>
#include <cstring>
#include "mystring2.h"

using namespace std;

template <class T>
class Vector {
public:

    class VectIter {
        friend class Vector<T>;

    private:
        Vector<T> *v; // points to a vector object of type T
        int index;    // subscript of the current element

    public:
        VectIter(Vector<T>& x);

        T operator++();
        // PROMISES: increments the iterator index and returns the value
        //               at the new index. Wraps to zero after the last element.

        T operator++(int);
        // PROMISES: returns the current value, then increments the iterator
        //               index. Wraps to zero after the last element.

        T operator--();
        // PROMISES: decrements the iterator index and returns the value
        //               at the new index. Wraps to the last element below zero.

        T operator--(int);
        // PROMISES: returns the current value, then decrements the iterator
        //               index. Wraps to the last element below zero.

        T& operator*();
        // PROMISES: returns the value at the current index position.
    };

    Vector(int sz);
    ~Vector();

    T& operator[](int i);
    // PROMISES: returns the ith array element so it can be read or changed.

    void ascending_sort();
    // PROMISES: sorts the vector values in ascending order.

private:
    T *array;
    int size;
    void swap(T&, T&);
};

template <class T>
void Vector<T>::ascending_sort()
{
    for(int i = 0; i < size - 1; i++)
        for(int j = i + 1; j < size; j++)
            if(array[i] > array[j])
                swap(array[i], array[j]);
}

template <class T>
void Vector<T>::swap(T& a, T& b)
{
    T tmp = a;
```

Exercise B - File: iterator.cpp (continued)

```
a = b;
b = tmp;
}

template <class T>
T& Vector<T>::VectIter::operator*()
{
    return v->array[index];
}

template <class T>
Vector<T>::VectIter::VectIter(Vector<T>& x)
{
    v = &x;
    index = 0;
}

template <class T>
T Vector<T>::VectIter::operator++()
{
    index++;
    if(index >= v->size)
        index = 0;

    return v->array[index];
}

template <class T>
T Vector<T>::VectIter::operator++(int)
{
    T value = v->array[index];

    index++;
    if(index >= v->size)
        index = 0;

    return value;
}

template <class T>
T Vector<T>::VectIter::operator--()
{
    index--;
    if(index < 0)
        index = v->size - 1;

    return v->array[index];
}

template <class T>
T Vector<T>::VectIter::operator--(int)
{
    T value = v->array[index];

    index--;
    if(index < 0)
        index = v->size - 1;

    return value;
}

template <class T>
Vector<T>::Vector(int sz)
{
    size = sz;
    array = new T[sz];
    assert(array != NULL);
}
```

Exercise B - File: iterator.cpp (continued)

```
}

template <class T>
Vector<T>::~~Vector()
{
    delete [] array;
    array = NULL;
}

template <class T>
T& Vector<T>::operator[](int i)
{
    return array[i];
}

// Specialization required for C-strings. Comparing const char* with > would
// compare pointer addresses rather than the text stored in the strings.
template <>
void Vector<const char*>::ascending_sort()
{
    for(int i = 0; i < size - 1; i++)
        for(int j = i + 1; j < size; j++)
            if(strcmp(array[i], array[j]) > 0)
                swap(array[i], array[j]);
}

int main()
{
    Vector<int> x(3);
    x[0] = 999;
    x[1] = -77;
    x[2] = 88;

    Vector<int>::VectIter iter(x);

    cout << "\n\nThe first element of vector x contains: " << *iter;

    // This section is now included for compilation and testing.
    #if 1
        cout << "\n\nTesting an <int> Vector: " << endl;

        cout << "\n\nTesting sort";
        x.ascending_sort();

        for(int i = 0; i < 3; i++)
            cout << endl << iter++;

        cout << "\n\nTesting Prefix --:";
        for(int i = 0; i < 3; i++)
            cout << endl << --iter;

        cout << "\n\nTesting Prefix ++:";
        for(int i = 0; i < 3; i++)
            cout << endl << ++iter;

        cout << "\n\nTesting Postfix --:";
        for(int i = 0; i < 3; i++)
            cout << endl << iter--;

        cout << endl;

        cout << "Testing a <Mystring> Vector: " << endl;
        Vector<Mystring> y(3);
        y[0] = "Bar";
        y[1] = "Foo";
        y[2] = "All";

        Vector<Mystring>::VectIter iters(y);

        cout << "\n\nTesting sort";
```

Exercise B - File: iterator.cpp (continued)

```
y.ascending_sort();

for(int i = 0; i < 3; i++)
    cout << endl << iters++;

cout << "\n\nTesting Prefix --:";
for(int i = 0; i < 3; i++)
    cout << endl << --iters;

cout << "\n\nTesting Prefix ++:";
for(int i = 0; i < 3; i++)
    cout << endl << ++iters;

cout << "\n\nTesting Postfix --:";
for(int i = 0; i < 3; i++)
    cout << endl << iters--;

cout << endl;
cout << "Testing a <char *> Vector: " << endl;
Vector<const char*> z(3);
z[0] = "Orange";
z[1] = "Pear";
z[2] = "Apple";

Vector<const char*>::VectIter iterchar(z);

cout << "\n\nTesting sort";
z.ascending_sort();

for(int i = 0; i < 3; i++)
    cout << endl << iterchar++;

#endif

cout << "\nProgram Terminated Successfully." << endl;

return 0;
}
```

Exercise B - Program Output (output.txt)

The first element of vector x contains: 999
Testing an <int> Vector:

Testing sort
-77
88
999

Testing Prefix --:
999
88
-77

Testing Prefix ++:
88
999
-77

Testing Postfix --
-77
999
88

Testing a <Mystring> Vector:

Testing sort
All
Bar
Foo

Testing Prefix --:
Foo
Bar
All

Testing Prefix ++:
Bar
Foo
All

Testing Postfix --
All
Foo
Bar

Testing a <char *> Vector:

Testing sort
Apple
Orange
Pear
Program Terminated Successfully.

Exercise C

Source files

mystring2.h
mystring2.cpp
customer.h
customer.cpp
lookupTable.h
mainLab3ExC.cpp

Output: output.txt

Exercise C - File: mystring2.h

```
//File: mystring2.h
// ENSF 480 - Lab 3

#ifndef MYSTRING_H
#define MYSTRING_H

#include <iostream>

class Mystring {
    friend std::ostream& operator <<(std::ostream& os, const Mystring& s);

public:
    Mystring();
    // PROMISES: Empty string object is created.

    Mystring(int n);
    // PROMISES: Creates an empty string with a total capacity of n.
    //           In other words, dynamically allocates n elements for
    //           charsM, sets lengthM to zero, and fills the first
    //           element of charsM with '\0'.

    Mystring(const char *s);
    // REQUIRES: s points to first char of a built-in string.
    // PROMISES: Mystring object is created by copying chars from s.

    ~Mystring();

    Mystring(const Mystring& source);

    Mystring& operator =(const Mystring& rhs);
    // REQUIRES: rhs is reference to a Mystring as a source.
    // PROMISES: Makes this object a copy of rhs.

    bool operator >(const Mystring& rhs) const;
    // PROMISES: Returns true if this string comes after rhs
    //           lexicographically.

    int length() const;
    // PROMISES: Return value is number of chars in charsM.

    char get_char(int pos) const;
    // REQUIRES: pos >= 0 && pos < length()
    // PROMISES: Return value is char at position pos.

    const char * c_str() const;
    // PROMISES: Return value points to first char in built-in string
    //           containing the chars of the string object.

    void set_char(int pos, char c);
    // REQUIRES: pos >= 0 && pos < length(), c != '\0'
    // PROMISES: Character at position pos is set equal to c.

    Mystring& append(const Mystring& other);
    // PROMISES: Concatenates other to the end of this string.

    void set_str(char* s);
    // REQUIRES: s is a valid C++ string of characters (a built-in string).
    // PROMISES: Copies s into charsM.

private:
    int lengthM;
    char* charsM;
    void memory_check(char* s);
};

#endif
```

Exercise C - File: mystring2.cpp

```
// mystring2.cpp
// ENSF 480 - Lab 3
// M. Moussavi

#include "mystring2.h"
#include <string.h>
#include <iostream>

using namespace std;

Mystring::Mystring()
{
    charsM = new char[1];
    charsM[0] = '\0';
    lengthM = 0;
}

Mystring::Mystring(const char *s)
    : lengthM((int)strlen(s))
{
    charsM = new char[lengthM + 1];
    strcpy(charsM, s);
}

Mystring::Mystring(int n)
    : lengthM(0), charsM(new char[n])
{
    charsM[0] = '\0';
}

Mystring::Mystring(const Mystring& source)
    : lengthM(source.lengthM), charsM(new char[source.lengthM + 1])
{
    strcpy(charsM, source.charsM);
}

Mystring::~Mystring()
{
    delete [] charsM;
}

int Mystring::length() const
{
    return lengthM;
}

char Mystring::get_char(int pos) const
{
    if(pos < 0 || pos >= length()) {
        cerr << "\nERROR: get_char: the position is out of boundary."
    }

    return charsM[pos];
}

const char * Mystring::c_str() const
{
    return charsM;
}

void Mystring::set_char(int pos, char c)
{
    if(pos < 0 || pos >= length()) {
        cerr << "\nset_char: the position is out of boundary."
        << " Nothing was changed.";
        return;
    }

    if(c == '\0') {
        cerr << "\nset_char: char c is empty."
        << " Nothing was changed.";
        return;
    }
}
```

Exercise C - File: mystring2.cpp (continued)

```
    charsM[pos] = c;
}

Mystring& Mystring::operator =(const Mystring& S)
{
    if(this == &S)
        return *this;

    delete [] charsM;
    lengthM = (int)strlen(S.charsM);
    charsM = new char[lengthM + 1];
    strcpy(charsM, S.charsM);
    return *this;
}

bool Mystring::operator >(const Mystring& rhs) const
{
    return strcmp(charsM, rhs.charsM) > 0;
}

Mystring& Mystring::append(const Mystring& other)
{
    char *tmp = new char[lengthM + other.lengthM + 1];
    strcpy(tmp, charsM);
    strcat(tmp, other.charsM);

    lengthM += other.lengthM;
    delete [] charsM;
    charsM = tmp;

    return *this;
}

void Mystring::set_str(char* s)
{
    delete [] charsM;
    lengthM = (int)strlen(s);
    charsM = new char[lengthM + 1];
    strcpy(charsM, s);
}

std::ostream& operator <<(std::ostream& os, const Mystring& s)
{
    return os << s.charsM;
}
```

Exercise C - File: customer.h

```
// ENSF 480 - Lab 3, Ex C
// M. Moussavi
#ifndef CUSTOMER
#define CUSTOMER
#include <iostream>

#include "mystring2.h"

using namespace std;

class Customer{
    friend ostream& operator << (ostream&, const Customer& c);

public:
    Customer(const Mystring& fn, const Mystring& ln, const Mystring& ad, const Mystring& ph);
    const char* getFname() const;
    const char* getLname()const;
    const char* getAddress()const;
    const char* getPhone()const;
    void setFname(const char*s);
    void setLname(const char*s);
    void setAddress(const char*s);
    void setPhone(const char*s);

private:
    Mystring fname;
    Mystring lname;
    Mystring address;
    Mystring phone;
};

#endif
```

Exercise C - File: customer.cpp

```
// ENSF 480 - Lab 3, Ex C
// M. Moussavi

#include "customer.h"
#include <iostream>

using namespace std;

Customer::Customer(const Mystring& fn, const Mystring& ln, const Mystring& ad, const Mystring& ph):fname(fn),
lname(ln), address(ad), phone(ph)
{
}

const char* Customer::getFname()const
{
    return fname.c_str();
}

const char* Customer::getLname()const
{
    return lname.c_str();
}

const char* Customer::getAddress()const
{
    return address.c_str();
}

const char* Customer::getPhone()const
{
    return phone.c_str();
}

void Customer::setFname(const char*s)
{
    Mystring tmp(s);
    fname=tmp;
}

void Customer::setLname(const char*s)
{
    Mystring tmp(s);
    lname=tmp;
}

void Customer::setAddress(const char*s)
{
    Mystring tmp(s);
    address=tmp;
}

void Customer::setPhone(const char*s)
{
    Mystring tmp(s);
    phone=tmp;
}

ostream& operator << (ostream& os, const Customer& c)
{
    os<< "Name: " << c.getFname();
    os<< " " << c.getLname();
    os<< ". Address: " << c.getAddress();
    os<< ". Phone: " << c.getPhone();
    return os;
}
```

Exercise C - File: lookupTable.h

```
// LookupTable.h
// ENSF 480 - Lab 3, Ex C

#ifndef LOOKUPTABLE_H
#define LOOKUPTABLE_H

#include <assert.h>
#include <cstring>
#include <iostream>
#include "customer.h"

using namespace std;

// Generic key comparison helpers.
// The general versions use the key type's relational operators.
template <class Key>
bool key_less(const Key& a, const Key& b)
{
    return a < b;
}

template <class Key>
bool key_equal(const Key& a, const Key& b)
{
    return a == b;
}

// C-string keys must be compared by their contents, not pointer addresses.
inline bool key_less(const char* const& a, const char* const& b)
{
    return strcmp(a, b) < 0;
}

inline bool key_equal(const char* const& a, const char* const& b)
{
    return strcmp(a, b) == 0;
}

// Mystring does not define all relational operators, so compare its C strings.
inline bool key_less(const Mystring& a, const Mystring& b)
{
    return strcmp(a.c_str(), b.c_str()) < 0;
}

inline bool key_equal(const Mystring& a, const Mystring& b)
{
    return strcmp(a.c_str(), b.c_str()) == 0;
}

// A generic key/datum pair.
template <class Key, class Datum>
struct Pair
{
    Pair(const Key& keyA, const Datum& datumA)
        : key(keyA), datum(datumA)
    {
    }

    Key key;
    Datum datum;
};

template <class Key, class Datum>
class LookupTable;

// A node used internally by LookupTable.
template <class Key, class Datum>
class LT_Node
{
    friend class LookupTable<Key, Datum>;

private:
    Pair<Key, Datum> pairM;
    LT_Node<Key, Datum>* nextM;
};
```

Exercise C - File: lookupTable.h (continued)

```
LT_Node(const Pair<Key, Datum>& pairA, LT_Node<Key, Datum>* nextA)
: pairM(pairA), nextM(nextA)
{
}
};

template <class Key, class Datum>
class LookupTable
{
public:
    class Iterator
    {
        friend class LookupTable<Key, Datum>;

    private:
        LookupTable<Key, Datum>* LT;

    public:
        Iterator() : LT(0) {}
        Iterator(LookupTable<Key, Datum>& x) : LT(&x) {}

        const Datum& operator*()
        {
            assert(LT != 0 && LT->cursor_ok());
            return LT->cursor_datum();
        }

        // These retain the behaviour expected by the supplied test loop:
        // return the current datum, then advance the table cursor.
        const Datum& operator++()
        {
            assert(LT != 0 && LT->cursor_ok());
            const Datum& x = LT->cursor_datum();
            LT->step_fwd();
            return x;
        }

        const Datum& operator++(int)
        {
            assert(LT != 0 && LT->cursor_ok());
            const Datum& x = LT->cursor_datum();
            LT->step_fwd();
            return x;
        }

        int operator!()
        {
            return LT != 0 && LT->cursor_ok();
        }

        void step_fwd()
        {
            assert(LT != 0 && LT->cursor_ok());
            LT->step_fwd();
        }
    };

};

LookupTable()
: sizeM(0), headM(0), cursorM(0)
{
}

LookupTable(const LookupTable<Key, Datum>& source)
: sizeM(0), headM(0), cursorM(0)
{
    copy(source);
}

LookupTable<Key, Datum>& operator=(const LookupTable<Key, Datum>& rhs)
{
    if (this != &rhs) {
        destroy();
        copy(rhs);
    }
}
```

Exercise C - File: lookupTable.h (continued)

```
    }
    return *this;
}

~LookupTable()
{
    destroy();
}

LookupTable<Key, Datum>& begin()
{
    cursorM = headM;
    return *this;
}

int size() const
{
    return sizeM;
}

int cursor_ok() const
{
    return cursorM != 0;
}

const Key& cursor_key() const
{
    assert(cursor_ok());
    return cursorM->pairM.key;
}

const Datum& cursor_datum() const
{
    assert(cursor_ok());
    return cursorM->pairM.datum;
}

void insert(const Pair<Key, Datum>& pairA)
{
    // Add a new node at the head.
    if (headM == 0 || key_less(pairA.key, headM->pairM.key)) {
        headM = new LT_Node<Key, Datum>(pairA, headM);
        sizeM++;
    }
    // Overwrite the datum at the head.
    else if (key_equal(pairA.key, headM->pairM.key)) {
        headM->pairM.datum = pairA.datum;
    }
    else {
        LT_Node<Key, Datum>* before = headM;
        LT_Node<Key, Datum>* after = headM->nextM;

        while (after != 0 && key_less(after->pairM.key, pairA.key)) {
            before = after;
            after = after->nextM;
        }

        if (after != 0 && key_equal(pairA.key, after->pairM.key)) {
            after->pairM.datum = pairA.datum;
        }
        else {
            before->nextM = new LT_Node<Key, Datum>(pairA, after);
            sizeM++;
        }
    }
}

cursorM = 0;
}

void remove(const Key& keyA)
{
    if (headM == 0 || key_less(keyA, headM->pairM.key)) {
        cursorM = 0;
        return;
    }
}
```

Exercise C - File: lookupTable.h (continued)

```
}

LT_Node<Key, Datum>* doomed_node = 0;

if (key_equal(keyA, headM->pairM.key)) {
    doomed_node = headM;
    headM = headM->nextM;
    sizeM--;
}
else {
    LT_Node<Key, Datum>* before = headM;
    LT_Node<Key, Datum>* maybe_doomed = headM->nextM;

    while (maybe_doomed != 0 && key_less(maybe_doomed->pairM.key, keyA)) {
        before = maybe_doomed;
        maybe_doomed = maybe_doomed->nextM;
    }

    if (maybe_doomed != 0 && key_equal(maybe_doomed->pairM.key, keyA)) {
        doomed_node = maybe_doomed;
        before->nextM = maybe_doomed->nextM;
        sizeM--;
    }
}

delete doomed_node;
cursorM = 0;
}

void find(const Key& keyA)
{
    LT_Node<Key, Datum>* ptr = headM;

    while (ptr != 0 && !key_equal(ptr->pairM.key, keyA))
        ptr = ptr->nextM;

    cursorM = ptr;
}

void go_to_first()
{
    cursorM = headM;
}

void step_fwd()
{
    assert(cursor_ok());
    cursorM = cursorM->nextM;
}

void make_empty()
{
    destroy();
}

private:
int sizeM;
LT_Node<Key, Datum>* headM;
LT_Node<Key, Datum>* cursorM;

void destroy()
{
    while (headM != 0) {
        LT_Node<Key, Datum>* doomed = headM;
        headM = headM->nextM;
        delete doomed;
    }

    cursorM = 0;
    sizeM = 0;
}

void copy(const LookupTable<Key, Datum>& source)
{

```

Exercise C - File: lookupTable.h (continued)

```
sizeM = 0;
headM = 0;
cursorM = 0;

LT_Node<Key, Datum>* tail = 0;

for (LT_Node<Key, Datum>* p = source.headM; p != 0; p = p->nextM) {
    LT_Node<Key, Datum>* newNode = new LT_Node<Key, Datum>(p->pairM, 0);

    if (headM == 0)
        headM = newNode;
    else
        tail->nextM = newNode;

    tail = newNode;
    sizeM++;

    if (source.cursorM == p)
        cursorM = newNode;
}
};

// Generic stream output for any LookupTable whose key and datum are streamable.
template <class Key, class Datum>
ostream& operator<<(ostream& os, const LookupTable<Key, Datum>& lt)
{
    if (lt.cursor_ok())
        os << lt.cursor_key() << " " << lt.cursor_datum();
    else
        os << "Not Found.";

    return os;
}

#endif
```

Exercise C - File: mainLab3ExC.cpp

```
// ENSF 480 - Lab 3, Ex C
// M. Moussavi

#include <assert.h>
#include <cstring>
#include <iostream>
#include "lookupTable.h"
#include "customer.h"

using namespace std;

template <class Key, class Datum>
void print(LookupTable<Key, Datum>& lt)
{
    if (lt.size() == 0)
        cout << " Table is EMPTY.\n";

    for (lt.go_to_first(); lt.cursor_ok(); lt.step_fwd())
        cout << lt << endl;
}

template <class Key, class Datum>
void try_to_find(LookupTable<Key, Datum>& lt, const Key& key)
{
    lt.find(key);

    if (lt.cursor_ok())
        cout << "\nFound key:" << lt;
    else
        cout << "\nSorry, I couldn't find key: " << key << " in the table.\n";
}

void test_Customer();
void test_String();
void test_integer();

int main()
{
    test_Customer();
    test_String();
    test_integer();

    cout << "\n\nProgram terminated successfully.\n\n";
    return 0;
}

void test_Customer()
{
    cout << "\nCreating and testing Customers LookupTable <int, Customer> ..... \n";
    LookupTable<int, Customer> lt;

    Customer a("Joe", "Morrison", "11 St. Calgary.", "(403)-1111-123333");
    Customer b("Jack", "Lewis", "12 St. Calgary.", "(403)-1111-123334");
    Customer c("Tim", "Hardy", "13 St. Calgary.", "(403)-1111-123335");

    lt.insert(Pair<int, Customer>(8002, a));
    lt.insert(Pair<int, Customer>(8004, c));
    lt.insert(Pair<int, Customer>(8001, b));

    assert(lt.size() == 3);
    lt.remove(8004);
    assert(lt.size() == 2);

    cout << "\nPrinting table after inserting 3 new keys and 1 removal...\n";
    print(lt);

    cout << "\nLet's look up some names ... \n";
    try_to_find(lt, 8001);
    try_to_find(lt, 8000);

    cout << "\nTesting and using iterator ... \n";
    typename LookupTable<int, Customer>::Iterator it = lt.begin();
    cout << "\nThe first node contains: " << *it << endl;
}
```

Exercise C - File: mainLab3ExC.cpp (continued)

```
while (!it)
    cout << ++it << endl;

lt.go_to_first();
lt.step_fwd();
LookupTable<int, Customer> clt(lt);
assert(strcmp(clt.cursor_datum().getFname(), "Joe") == 0);

cout << "\nTest copying: keys should be 8001, and 8002\n";
print(clt);
lt.remove(8002);

clt = lt;

cout << "\nTest assignment operator: key should be 8001\n";
print(clt);

lt.make_empty();
cout << "\nPrinting table for the last time: Table should be empty...\n";
print(lt);

cout << "****---Finished tests on LookupTable <int, Customer>-----***\n";
}

void test_String()
{
    cout << "\nCreating and testing LookupTable <int, Mystring> ..... \n";
    LookupTable<int, Mystring> lt;

    Mystring a("I am an ENEL-409 student.");
    Mystring b("C++ is a powerful language for engineers but it's not easy.");
    Mystring c("Winter 2004");

    lt.insert(Pair<int, Mystring>(8002, a));
    lt.insert(Pair<int, Mystring>(8001, b));
    lt.insert(Pair<int, Mystring>(8004, c));

    assert(lt.size() == 3);
    lt.remove(8004);
    assert(lt.size() == 2);

    cout << "\nPrinting table after inserting 3 new keys and 1 removal...\n";
    print(lt);

    cout << "\nLet's look up some names ... \n";
    try_to_find(lt, 8001);
    try_to_find(lt, 8000);

    typename LookupTable<int, Mystring>::Iterator it = lt.begin();
    cout << "\nThe first node contains: " << *it << endl;

    while (!it)
        cout << ++it << endl;

    lt.go_to_first();
    lt.step_fwd();
    LookupTable<int, Mystring> clt(lt);
    assert(strcmp(clt.cursor_datum().c_str(), "I am an ENEL-409 student.") == 0);

    cout << "\nTest copying: keys should be 8001, and 8002\n";
    print(clt);
    lt.remove(8002);

    clt = lt;

    cout << "\nTest assignment operator: key should be 8001\n";
    print(clt);

    lt.make_empty();
    cout << "\nPrinting table for the last time: Table should be empty ... \n";
    print(lt);

    cout << "****---Finished Lab 3 tests on <int, Mystring>-----***\n";
}
```

Exercise C - File: mainLab3ExC.cpp (continued)

```
void test_integer()
{
    cout << "\nCreating and testing LookupTable <int, int> ....\n";
    LookupTable<int, int> lt;

    lt.insert(Pair<int, int>(8002, 9999));
    lt.insert(Pair<int, int>(8001, 8888));
    lt.insert(Pair<int, int>(8004, 8888));

    assert(lt.size() == 3);
    lt.remove(8004);
    assert(lt.size() == 2);

    cout << "\nPrinting table after inserting 3 new keys and 1 removal...\n";
    print(lt);

    cout << "\nLet's look up some names ...\n";
    try_to_find(lt, 8001);
    try_to_find(lt, 8000);

    typename LookupTable<int, int>::Iterator it = lt.begin();
    cout << "\nThe first node contains: " << *it << endl;

    while (!it)
        cout << ++it << endl;

    lt.go_to_first();
    lt.step_fwd();
    LookupTable<int, int> clt(lt);
    assert(clt.cursor_datum() == 9999);

    cout << "\nTest copying: keys should be 8001, and 8002\n";
    print(clt);
    lt.remove(8002);

    clt = lt;

    cout << "\nTest assignment operator: key should be 8001\n";
    print(clt);

    lt.make_empty();
    cout << "\nPrinting table for the last time: Table should be empty ...\n";
    print(lt);

    cout << "****---Finished Lab 3 tests on <int, int>-----****\n";
}
```

Exercise C - Program Output (output.txt)

```
Creating and testing Customers LookupTable <int, Customer> .....

Printing table after inserting 3 new keys and 1 removal...
8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
8002 Nmae: Joe Morrison. Address: 11 St. Calgary.. Phone:: (403)-1111-123333

Let's look up some names ...

Found key:8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
Sorry, I couldn't find key: 8000 in the table.

Testing and using iterator ...

The first node contains: Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
Nmae: Joe Morrison. Address: 11 St. Calgary.. Phone:: (403)-1111-123333

Test copying: keys should be 8001, and 8002
8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334
8002 Nmae: Joe Morrison. Address: 11 St. Calgary.. Phone:: (403)-1111-123333

Test assignment operator: key should be 8001
8001 Nmae: Jack Lewis. Address: 12 St. Calgary.. Phone:: (403)-1111-123334

Printing table for the last time: Table should be empty...
Table is EMPTY.
***-----Finished tests on LookupTable <int, Customer>-----***

Creating and testing LookupTable <int, Mystring> .....

Printing table after inserting 3 new keys and 1 removal...
8001 C++ is a powerful language for engineers but it's not easy.
8002 I am an ENEL-409 student.

Let's look up some names ...

Found key:8001 C++ is a powerful language for engineers but it's not easy.
Sorry, I couldn't find key: 8000 in the table.

The first node contains: C++ is a powerful language for engineers but it's not easy.
C++ is a powerful language for engineers but it's not easy.
I am an ENEL-409 student.

Test copying: keys should be 8001, and 8002
8001 C++ is a powerful language for engineers but it's not easy.
8002 I am an ENEL-409 student.

Test assignment operator: key should be 8001
8001 C++ is a powerful language for engineers but it's not easy.

Printing table for the last time: Table should be empty ...
Table is EMPTY.
***-----Finished Lab 3 tests on <int, Mystring>-----***

Creating and testing LookupTable <int, int> .....

Printing table after inserting 3 new keys and 1 removal...
8001 8888
8002 9999

Let's look up some names ...

Found key:8001 8888
Sorry, I couldn't find key: 8000 in the table.

The first node contains: 8888
8888
9999

Test copying: keys should be 8001, and 8002
8001 8888
8002 9999
```

Exercise C - Program Output (output.txt) (continued)

Test assignment operator: key should be 8001
8001 8888

Printing table for the last time: Table should be empty ...
Table is EMPTY.

----Finished Lab 3 tests on <int, int>-----

Program terminated successfully.